TEMA 3. Excepciones

1. Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función `raiz` ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

Respuesta

En C no existe un mecanismo de excepciones incorporado en el lenguaje, por lo que el control de errores debe realizarse de forma manual mediante convenciones. La función que detecta el error no puede simplemente "avisar" al llamador de forma automática; es responsabilidad del programador diseñar un protocolo para comunicar que algo salió mal. A continuación se presentan dos opciones habituales.

Opción 1: Devolver un valor especial que indique error. Se utiliza el propio valor de retorno de la función para señalar que se ha producido un error. Por ejemplo, se puede devolver un número negativo (como `-1.0`) si la entrada no es válida, dado que una raíz cuadrada real nunca es negativa:

```c
#include <stdio.h>
#include <math.h>

double raiz(double n) {
    if (n < 0) {
        return -1.0; // Valor especial que indica error
    }
    return sqrt(n);
}

int main() {
    double resultado = raiz(-4.0);
    if (resultado < 0) {
        printf("Error: no se puede calcular la raíz de un número
negativo.\n");
    } else {
        printf("Resultado: %f\n", resultado);
    }
    return 0;
}
```

Opción 2: Utilizar un parámetro de salida para el resultado y devolver un código de error. En este enfoque, el valor de retorno de la función se reserva exclusivamente para indicar si hubo éxito o error (por ejemplo, `0` para éxito y `1` para error), y el resultado real se devuelve a través de un puntero pasado como argumento:

```c
#include <stdio.h>
#include <math.h>

int raiz(double n, double *resultado) {
    if (n < 0) {
        return 1; // Código de error
    }
    *resultado = sqrt(n);
    return 0; // Éxito
}

int main() {
    double resultado;
    int error = raiz(-4.0, &resultado);
    if (error) {
        printf("Error: no se puede calcular la raíz de un número
negativo.\n");
    } else {
        printf("Resultado: %f\n", resultado);
    }
    return 0;
}
```

Ambas opciones tienen un inconveniente fundamental: el llamador puede ignorar el código de error (ya sea el valor de retorno especial o el código numérico), lo que haría que el programa continuara ejecutándose con datos incorrectos sin saberlo. Además, el significado del error queda poco claro sin documentación adicional, ya que un número como `-1` o `1` no describe por sí mismo qué ha fallado.

2. Brevemente ¿Qué es una **"excepción"**? ¿Con qué objetivo las usa un programador cuando implementa funciones o cuando las llama?

Respuesta

Una excepción es un mecanismo que ofrecen ciertos lenguajes de programación (como Java) para señalar que se ha producido una situación anormal o inesperada durante la ejecución de un programa. En lugar de devolver valores especiales o códigos de error como se hace en C, cuando se detecta un error se "lanza" una excepción, lo que interrumpe el flujo normal del programa y transfiere el control a un punto donde ese error pueda ser tratado adecuadamente.

Cuando un programador implementa una función, utiliza las excepciones para avisar al código llamador de que algo ha ido mal, sin necesidad de "contaminar" el valor de retorno con códigos de error. De este modo, la función puede devolver siempre un resultado válido si todo va bien, y si algo falla, lanza una excepción que obliga al llamador a ocuparse del problema.

Desde el punto de vista del llamador, las excepciones permiten separar claramente el código que realiza la lógica principal del código que maneja los errores. Esto mejora la legibilidad y la robustez del programa, ya que no se puede ignorar inadvertidamente un error: si una excepción no se captura, el programa se detendrá, lo cual es preferible a seguir ejecutándose con datos corruptos.

3. Reescribe el mismo ejemplo de raiz, pero en Java, metiendo ese método en una clase `Calculadora` y llama a dicho método desde el método `main`, mostrando cómo se puede controlar desde fuera.

Respuesta

En Java, en lugar de devolver un valor especial o un código de error, se puede lanzar una excepción cuando el argumento no es válido. La clase `Calculadora` contiene el método `raiz` que, si recibe un número negativo, lanza una excepción de tipo `IllegalArgumentException`. Así, el propio lenguaje se encarga de interrumpir el flujo normal y forzar al llamador a gestionar el error.

Desde el método `main`, se utiliza un bloque `try-catch` para intentar ejecutar el código que podría fallar y, si se produce la excepción, capturarla y mostrar un mensaje al usuario. De esta forma, el control de errores queda separado de la lógica principal:

```
public class Calculadora {

    public static double raiz(double n) {
        if (n < 0) {
            throw new IllegalArgumentException("No se puede calcular la
raíz de un número negativo: " + n);
        }
        return Math.sqrt(n);
    }

    public static void main(String[] args) {
        try {
            double resultado = raiz(-4.0);
            System.out.println("Resultado: " + resultado);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Nótese la diferencia respecto a C: aquí no es necesario comprobar manualmente un valor de retorno. Si `raiz` lanza la excepción, la línea `System.out.println("Resultado: ...")` nunca se ejecuta, y el control salta directamente al bloque `catch`. Esto hace que sea imposible usar accidentalmente un resultado inválido.

4. ¿Qué es **"lanzar"** una excepción? ¿Qué es **"controlar"** o **"capturar"** una excepción? ¿Qué es que se **"propague"** una excepción? ¿Qué le va ocurriendo a las funciones en la pila de llamadas por donde se va propagando la excepción? ¿Las funciones que no la controlan se reanudan después de alguna forma? Explica con el mismo ejemplo anterior en Java de la raíz cuadrada.

Respuesta

Lanzar una excepción significa crear un objeto excepción y emitirlo mediante la palabra clave `throw`. En el ejemplo de la `Calculadora`, cuando `raiz` recibe un número negativo, ejecuta `throw new IllegalArgumentException(...)`. En ese instante, el método `raiz` se interrumpe inmediatamente y no ejecuta ninguna línea posterior al `throw`.

Capturar o **controlar** una excepción significa interceptarla mediante un bloque `try-catch` para tratarla de forma adecuada. En el ejemplo, el `main` envuelve la llamada a `raiz(-4.0)` dentro de un `try`, y si la excepción se produce, el bloque `catch (IllegalArgumentException e)` la recibe y ejecuta el código de manejo del error (en este caso, imprimir el mensaje).

Propagarse significa que, si la función que lanza la excepción no la captura ella misma, la excepción "sube" por la pila de llamadas hacia la función que la invocó, luego hacia la que invocó a esa, y así sucesivamente. Cada función por la que pasa la excepción se interrumpe inmediatamente en el punto donde hizo la llamada, sin ejecutar el resto de su código. **No se reanudan**: quedan abortadas definitivamente. Por ejemplo, si hubiera una función intermedia `calcular` que llamara a `raiz` sin `try-catch`, y `main` llamara a `calcular`:

```
public class Calculadora {

    public static double raiz(double n) {
        if (n < 0) {
            throw new IllegalArgumentException("Número negativo: " + n);
        }
        return Math.sqrt(n);
    }

    public static void calcular(double n) {
        double resultado = raiz(n);           // Si raiz lanza excepción,
se interrumpe aquí
        System.out.println("Resultado: " + resultado);  // Esta línea
NUNCA se ejecuta
    }

    public static void main(String[] args) {
        try {
            calcular(-4.0);                   // calcular se interrumpe,
la excepción llega aquí
            System.out.println("Fin del cálculo");       // Esta línea
NUNCA se ejecuta
        } catch (IllegalArgumentException e) {
            System.out.println("Error capturado en main: " +
e.getMessage());
        }
    }
}
```

En este ejemplo, `raiz` lanza la excepción, `calcular` no la captura y por tanto se interrumpe (su `println` nunca se ejecuta), y la excepción sigue subiendo hasta el `catch` del `main`. Ninguna de las funciones intermedias se reanuda tras la propagación; simplemente dejan de ejecutarse.

5. ¿Qué ventajas tiene frente a C, la **"propagación natural"** de las excepciones a través de la pila (*stack*) de llamadas?

Respuesta

En C, si se tiene una cadena de llamadas (por ejemplo, `main` → `funcionA` → `funcionB` → `funcionC`), y `funcionC` detecta un error, cada función intermedia debe comprobar explícitamente el código de error devuelto por la función que llamó y reenviarlo hacia arriba. Esto implica escribir comprobaciones de error (`if`) en cada nivel de la pila, lo que genera código repetitivo, propenso a errores y difícil de mantener. Si en algún nivel se olvida comprobar el valor de retorno, el error se pierde silenciosamente.

Con las excepciones, la propagación es automática. Si `funcionC` lanza una excepción y ni `funcionB` ni `funcionA` la capturan, esta sube naturalmente por la pila hasta llegar al primer bloque `try-catch` que pueda manejarla, sin necesidad de que cada función intermedia tenga código de reenvío de errores. Esto simplifica enormemente el código de las funciones intermedias, que solo necesitan ocuparse de su lógica principal.

Además, en C es fácil que un programador ignore un código de error (no hay nada que lo impida), mientras que con las excepciones, si nadie las captura, el programa se detiene mostrando información del error. Esto garantiza que los errores nunca pasan desapercibidos y obliga a tratarlos de forma consciente.

6. En orientación a objetos, ¿las excepciones suelen ser objetos? ¿Qué ventajas tiene esto en términos de encapsulación? ¿Podemos entonces crear excepciones personalizadas?

Respuesta

Sí, en lenguajes orientados a objetos como Java, las excepciones son objetos. Toda excepción es una instancia de una clase que hereda de `Throwable` (habitualmente de `Exception` o de `RuntimeException`). Al ser objetos, las excepciones pueden contener atributos y métodos que encapsulan toda la información relevante sobre el error ocurrido, como un mensaje descriptivo, el momento en que se produjo, o datos específicos del contexto.

Esto ofrece una gran ventaja en términos de encapsulación: en lugar de manejar un simple código numérico como en C (donde un `1` o un `-1` no aportan contexto), una excepción como objeto puede llevar consigo el mensaje de error, la traza de la pila de llamadas (stack trace), e incluso atributos personalizados. Toda esa información queda empaquetada dentro del propio objeto, manteniendo el principio de encapsulación.

Además, es posible crear excepciones personalizadas definiendo nuevas clases que hereden de `Exception` o `RuntimeException`. Por ejemplo, se podría crear una excepción `NumeroNegativoException` para el caso de la raíz cuadrada:

```java
public class NumeroNegativoException extends RuntimeException {
    private double numero;

    public NumeroNegativoException(double numero) {
        super("Número negativo no permitido: " + numero);
        this.numero = numero;
    }
```

```
    public double getNumero() {
        return numero;
    }
}
```

De esta forma, se puede lanzar y capturar una excepción con un nombre descriptivo y con datos adicionales encapsulados en ella, lo que hace el código mucho más expresivo y mantenible que un simple código de error numérico.

7. En relación con las ventajas de la encapsulación, comparando el ejemplo en C con Java. ¿Qué **información esencial** lleva cualquier **objeto excepción** que es muy útil tener cuando se llega a un manejador?

Respuesta

Cualquier objeto excepción en Java lleva consigo, como mínimo, dos piezas de información fundamentales: un **mensaje descriptivo** (`message`) y la **traza de la pila de llamadas** (`stack trace`). El mensaje se establece al crear la excepción (por ejemplo, `"Número negativo no permitido: -4.0"`) y se obtiene con `e.getMessage()`. La traza de la pila es un registro completo de toda la cadena de llamadas a métodos que estaban activas en el momento en que se lanzó la excepción, incluyendo los nombres de las clases, los métodos y los números de línea exactos.

En C, cuando se detecta un error mediante un código de retorno, solo se sabe que "hubo un error", pero no se tiene automáticamente información de en qué línea se produjo, ni cuál fue la secuencia de llamadas que condujo a él. El programador tendría que imprimir manualmente esa información, lo que raramente se hace de forma completa.

En Java, cuando una excepción llega a un manejador (`catch`), se puede llamar a `e.printStackTrace()` para ver toda la traza, lo cual es extremadamente útil para depuración. Por ejemplo, una traza podría mostrar algo como:

```
java.lang.IllegalArgumentException: Número negativo: -4.0
    at Calculadora.raiz(Calculadora.java:5)
    at Calculadora.calcular(Calculadora.java:11)
    at Calculadora.main(Calculadora.java:17)
```

Esta información permite localizar inmediatamente dónde se originó el error y cómo se llegó hasta allí, algo que en C requeriría un esfuerzo manual considerable.

8. En Java, sobre el bloque **"try-catch"**, ¿se pueden tener más de un bloque `catch`? ¿cuántos bloques `catch` se ejecutan?

Respuesta

Sí, se pueden tener múltiples bloques `catch` asociados a un mismo `try`. Cada bloque `catch` especifica un tipo de excepción diferente, de modo que se pueden manejar distintos errores de distinta manera. Los

bloques `catch` se evalúan en orden, de arriba a abajo, y se ejecuta **únicamente el primero** cuyo tipo coincida con la excepción lanzada (o sea un supertipo de ella). Los demás bloques `catch` se ignoran.

Por este motivo, es importante colocar los tipos más específicos antes que los más generales. Si se pusiera un `catch (Exception e)` primero, capturaría cualquier excepción y los bloques `catch` posteriores nunca serían alcanzados (de hecho, el compilador de Java generaría un error por código inalcanzable). A continuación se muestra un ejemplo:

```
try {
    double resultado = raiz(Double.parseDouble(args[0]));
    System.out.println("Resultado: " + resultado);
} catch (NumberFormatException e) {
    System.out.println("Error: el argumento no es un número válido.");
} catch (IllegalArgumentException e) {
    System.out.println("Error: argumento ilegal - " + e.getMessage());
} catch (Exception e) {
    System.out.println("Error inesperado: " + e.getMessage());
}
```

En este ejemplo, si se lanza un `NumberFormatException`, solo se ejecuta el primer `catch`. Si se lanza un `IllegalArgumentException` (que no sea `NumberFormatException`), se ejecuta el segundo. Y si ocurriese cualquier otra excepción, se captura en el tercero. En ningún caso se ejecuta más de un bloque `catch`.

9. Si las excepciones producen rupturas en el código llamador, ¿cómo podemos garantizar que se ejecuta siempre finalmente un código necesario para cierre de ficheros, liberacion de recursos, antes de que continúe propagándose la excepción? Pon un ejemplo en Java con `finally`, tanto con `catch` como sin él.

Respuesta

Para garantizar que un bloque de código se ejecute siempre, tanto si ocurre una excepción como si no, Java ofrece el bloque `finally`. Este bloque se coloca después del `try` (y de los `catch`, si los hay) y su contenido se ejecuta **siempre**, independientemente de si se produjo una excepción, si se capturó, o incluso si el código del `try` finalizó normalmente. Esto es esencial para liberar recursos como ficheros abiertos, conexiones de red o de base de datos.

Ejemplo con `catch` y `finally`:

```
import java.io.*;

public class LectorFichero {
    public static void leerFichero(String ruta) {
        BufferedReader reader = null;
        try {
            reader = new BufferedReader(new FileReader(ruta));
            String linea = reader.readLine();
```

```
            System.out.println("Primera línea: " + linea);
        } catch (IOException e) {
            System.out.println("Error al leer el fichero: " +
e.getMessage());
        } finally {
            try {
                if (reader != null) {
                    reader.close();
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el fichero: " +
e.getMessage());
            }
            System.out.println("Bloque finally ejecutado: recurso
liberado.");
        }
    }
}
```

Ejemplo solo con `try-finally` (sin `catch`):

```
import java.io.*;

public class LectorFichero {
    public static void leerFichero(String ruta) throws IOException {
        BufferedReader reader = null;
        try {
            reader = new BufferedReader(new FileReader(ruta));
            String linea = reader.readLine();
            System.out.println("Primera línea: " + linea);
        } finally {
            try {
                if (reader != null) {
                    reader.close();
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el fichero: " +
e.getMessage());
            }
            System.out.println("Bloque finally ejecutado: recurso
liberado.");
        }
    }
}
```

En el segundo ejemplo, al no haber `catch`, si se produce una `IOException`, el bloque `finally` se ejecuta igualmente (cerrando el recurso), y luego la excepción continúa propagándose hacia arriba por la pila de llamadas. Esto permite garantizar la limpieza de recursos sin asumir la responsabilidad de manejar el error en ese nivel.

10. En Java, el bloque `finally` puede ir sin `catch`? ¿Se ejecuta siempre tanto si ocurre como si no ocurre una excepción? ¿Y si hay un `return` en medio del `try`?

Respuesta

Sí, como se vio en la pregunta anterior, el bloque `finally` puede ir directamente después de un `try`, sin ningún `catch`. La estructura `try-finally` es perfectamente válida en Java y se usa cuando se quiere garantizar la ejecución de un código de limpieza sin capturar la excepción (dejándola propagarse).

El bloque `finally` se ejecuta **siempre**, en todos estos escenarios: si el código del `try` termina normalmente, si se lanza una excepción (tanto si se captura como si no), e incluso si hay un `return` dentro del `try`. El único caso excepcional donde no se ejecuta es si se llama a `System.exit()`, que termina la máquina virtual abruptamente.

El caso del `return` merece atención especial. Si dentro del `try` hay un `return`, el bloque `finally` se ejecuta **antes** de que el método devuelva el valor. Esto se muestra en el siguiente ejemplo:

```
public class EjemploFinally {
    public static int obtenerValor() {
        try {
            System.out.println("Dentro del try");
            return 1;
        } finally {
            System.out.println("Dentro del finally (se ejecuta antes del
return)");
        }
    }

    public static void main(String[] args) {
        int valor = obtenerValor();
        System.out.println("Valor devuelto: " + valor);
    }
}
// Salida:
// Dentro del try
// Dentro del finally (se ejecuta antes del return)
// Valor devuelto: 1
```

Como se observa, aunque el `try` contiene un `return 1`, el bloque `finally` se ejecuta justo antes de que se devuelva el valor. Esto garantiza que los recursos se liberan correctamente en cualquier circunstancia.

11. En Java, qué son las excepciones **"controladas"** y las **"no controladas"**? ¿Qué papel juega `RuntimeException`? Pon un ejemplo de excepciones típicas controladas y no controladas que incluso nosotros mismos podríamos usar. Haz dos listas con 3 o 4 ejemplos de situación donde se suele preferir una excepción controlada y donde se suele preferir una excepción no controlada.

Respuesta

Las excepciones **controladas** (*checked exceptions*) son aquellas que el compilador obliga a gestionar. Si un método puede lanzar una excepción controlada, el código llamador está obligado a capturarla con `try-catch` o a declararla en su firma con `throws`. Son subclases de `Exception` pero **no** de `RuntimeException`. Ejemplos típicos: `IOException` (errores de entrada/salida), `FileNotFoundException` (fichero no encontrado), `SQLException` (errores de base de datos).

Las excepciones **no controladas** (*unchecked exceptions*) son aquellas que el compilador no obliga a capturar ni declarar. Son subclases de `RuntimeException`. Su papel es representar errores de programación o condiciones que no deberían ocurrir si el código está bien escrito. `RuntimeException` es la clase base de todas las excepciones no controladas. Ejemplos típicos: `NullPointerException` (acceso a referencia nula), `IllegalArgumentException` (argumento inválido), `ArrayIndexOutOfBoundsException` (índice fuera de rango), `ArithmeticException` (por ejemplo, división por cero).

Situaciones donde se suele preferir una excepción controlada:

1. Apertura de un fichero que podría no existir (`FileNotFoundException`).
2. Lectura o escritura en red, donde la conexión puede fallar (`IOException`).
3. Consultas a una base de datos que puede no estar disponible (`SQLException`).
4. Parseo de un documento XML o JSON con formato potencialmente incorrecto (`ParseException`).

Situaciones donde se suele preferir una excepción no controlada:

1. Recibir un argumento `null` cuando no se permite (`NullPointerException` o `IllegalArgumentException`).
2. Acceder a una posición fuera de un array (`ArrayIndexOutOfBoundsException`).
3. Pasar un valor fuera de rango a un método, como un número negativo a la raíz (`IllegalArgumentException`).
4. Intentar usar un objeto en un estado inválido, por ejemplo operar sobre una conexión ya cerrada (`IllegalStateException`).

La diferencia clave es que las controladas representan problemas **externos** al programa (ficheros, red, datos de entrada) que el programador no puede evitar, mientras que las no controladas representan **errores de programación** que deberían corregirse en el código.

12. ¿Qué es y para qué se usa `throws`? ¿Por qué es alternativa a capturar una excepción controlada?

Respuesta

La palabra clave `throws` se coloca en la firma de un método para declarar que dicho método puede lanzar una o varias excepciones controladas, y que no se responsabiliza de capturarlas. Es una forma de delegar la responsabilidad del manejo del error al método que realice la llamada. Su sintaxis es: `public void miMetodo() throws IOException, SQLException { ... }`.

Se usa `throws` cuando un método sabe que puede producirse una excepción controlada (por ejemplo, al abrir un fichero), pero considera que no es el lugar adecuado para manejarla. En lugar de envolver el código en un `try-catch`, declara la excepción en su firma con `throws`, y es el llamador quien deberá decidir qué

hacer: capturarla con `try-catch` o, a su vez, declararla también con `throws` para seguir propagándola hacia arriba.

Es una alternativa a capturar la excepción porque el compilador de Java exige que toda excepción controlada sea tratada de alguna manera. Para satisfacer al compilador, un método tiene exactamente dos opciones: o la captura con `try-catch`, o la declara con `throws`. Si se elige `throws`, la excepción se propaga automáticamente al llamador, que a su vez tendrá que elegir entre esas mismas dos opciones.

13. Pon un ejemplo en Java de firma de método que incluya `throws`, de una función que abre un fichero pero que declara que no le interesa menejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del `finally`.

Respuesta

A continuación se muestra un método `leerPrimeraLinea` que abre un fichero y lee su primera línea. El método declara con `throws FileNotFoundException` que no se responsabiliza de manejar el caso en que el fichero no exista, pero utiliza `finally` para asegurar que el recurso se cierra correctamente en cualquier caso:

```
import java.io.*;

public class LectorFichero {

    public static String leerPrimeraLinea(String ruta) throws
FileNotFoundException {
        BufferedReader reader = null;
        try {
            reader = new BufferedReader(new FileReader(ruta));
            return reader.readLine();
        } catch (IOException e) {
            // Captura errores de lectura, pero no FileNotFoundException
            // (FileNotFoundException se propaga por el throws)
            throw new RuntimeException("Error durante la lectura del
fichero", e);
        } finally {
            try {
                if (reader != null) {
                    reader.close();
                }
            } catch (IOException e) {
                System.err.println("Error al cerrar el fichero: " +
e.getMessage());
            }
        }
    }

    public static void main(String[] args) {
        try {
            String linea = leerPrimeraLinea("datos.txt");
            System.out.println("Primera línea: " + linea);
```

```
        } catch (FileNotFoundException e) {
            System.out.println("Error: el fichero no existe - " +
e.getMessage());
        }
    }
}
```

El método `leerPrimeraLinea` declara `throws FileNotFoundException` en su firma. Esto significa que si el fichero no existe, la excepción se propagará al llamador (`main`, en este caso), que es quien se encarga de capturarla con `try-catch`. Mientras tanto, se confía en el bloque `finally` para que el `BufferedReader` se cierre correctamente, independientemente de si se produjo una excepción o no.

Este patrón es muy habitual en programas reales: los métodos de bajo nivel que acceden a recursos no siempre saben cómo reaccionar ante un error (¿mostrar un mensaje?, ¿reintentar?, ¿usar un fichero alternativo?), así que delegan la decisión al llamador mediante `throws`.

14. ¿Podemos poner en `throws` excepciones no controladas, como `RuntimeException`? ¿Debería el método llamador entonces poner `try-catch` en ese caso? ¿Qué sentido tendría?

Respuesta

Sí, técnicamente es válido poner excepciones no controladas como `RuntimeException` o sus subclases en la cláusula `throws` de un método. Java no lo prohíbe. Sin embargo, a diferencia de las excepciones controladas, el compilador **no obliga** al método llamador a capturarlas ni a declararlas. Es decir, el llamador puede ignorar la declaración `throws RuntimeException` sin que el compilador produzca ningún error.

Entonces, ¿qué sentido tendría declararlas? El único propósito es **documentar** la intención del programador. Al poner `throws IllegalArgumentException` en la firma del método, se está comunicando al usuario de ese método que bajo ciertas condiciones puede lanzarse esa excepción. Es una forma de aviso que aparece en la API del método, similar a lo que se haría con un comentario Javadoc.

Sin embargo, en la práctica esta técnica no es muy habitual. La convención más común es documentar las excepciones no controladas mediante comentarios `@throws` en el Javadoc del método, en lugar de ponerlas en la firma con `throws`. El método llamador **no debería** necesitar un `try-catch` para excepciones no controladas, ya que estas suelen representar errores de programación que deben corregirse en el código, no capturarse en tiempo de ejecución.

15. ¿Cuándo se recomienda usar excepciones controladas, como `IOException`, y cuándo no controladas como `IllegalArgumentException`? ¿Existen en todos los lenguajes ambas opciones? En los que sólo existe una opción, ¿cuál es la más habitual?

Respuesta

Se recomienda usar excepciones **controladas** para representar problemas **recuperables** y causados por factores externos al programa, es decir, situaciones que pueden ocurrir aunque el código esté perfectamente escrito: un fichero que no existe, una conexión de red que falla, una base de datos que no

responde. En estos casos, tiene sentido obligar al programador a considerar cómo reaccionar ante el error, ya que son situaciones previsibles pero inevitables.

Las excepciones **no controladas** se recomiendan para errores de **programación**: pasar un argumento inválido (`IllegalArgumentException`), acceder a una referencia nula (`NullPointerException`), usar un índice fuera de rango (`ArrayIndexOutOfBoundsException`). Estos errores no deberían ocurrir si el código es correcto, y la solución no es capturarlos, sino corregir el código que los provoca.

No todos los lenguajes ofrecen ambas opciones. Java es uno de los pocos lenguajes que distingue entre excepciones controladas y no controladas. La mayoría de los lenguajes modernos (como C#, Python, Kotlin, JavaScript, C++) solo tienen **excepciones no controladas**, es decir, el compilador nunca obliga a capturar una excepción. Esta es la opción más habitual porque simplifica el código y evita la verbosidad que a veces generan las excepciones controladas, aunque a costa de perder la garantía de que el programador manejará ciertos errores.

16. ¿Tiene sentido lanzar excepciones dentro del `catch`? ¿Se puede relanzar la misma excepción capturada? ¿Cuándo tendría sentido hacer esto último? Pon ejemplos de ambos casos.

Respuesta

Sí, tiene todo el sentido lanzar excepciones dentro de un bloque `catch`. Es una práctica habitual por dos motivos principales: **transformar** la excepción en otra más adecuada para el nivel de abstracción actual, o **relanzar** la misma excepción tras haber realizado alguna acción previa (como registrar el error en un log).

Ejemplo de lanzar una excepción diferente (transformación): se captura una excepción de bajo nivel y se lanza otra de más alto nivel, más significativa para el llamador:

```
public class ServicioUsuarios {
    public static String cargarConfiguracion(String ruta) {
        try {
            BufferedReader reader = new BufferedReader(new
FileReader(ruta));
            return reader.readLine();
        } catch (IOException e) {
            // Se transforma la excepción técnica en una de dominio
            throw new RuntimeException("No se pudo cargar la configuración
del sistema", e);
        }
    }
}
```

Ejemplo de relanzar la misma excepción tras registrarla: a veces se quiere dejar constancia del error antes de propagarlo, sin asumir la responsabilidad de manejarlo completamente:

```
public class Procesador {
    public static void procesarDatos(String ruta) throws IOException {
        try {
```

```
            BufferedReader reader = new BufferedReader(new
FileReader(ruta));
            // ... procesamiento ...
        } catch (IOException e) {
            System.err.println("LOG: Error procesando " + ruta + " - " +
e.getMessage());
            throw e;  // Se relanza la misma excepción
        }
    }
}
```

Relanzar la misma excepción tiene sentido cuando el método actual quiere realizar una acción parcial (registrar el error, liberar un recurso específico, notificar algo), pero no es el responsable final de decidir qué hacer con el error. Tras la acción parcial, se relanza la excepción para que los niveles superiores de la pila tomen la decisión final.

17. ¿En qué consiste que una excepción sea la **"causa"** de otra excepción? Pon un ejemplo en Java, donde capturemos una excepción de bajo nivel y la encapsulemos en otra personalizada de alto nivel. Cuando una excepción sale por pantalla y tiene una causa, ¿se ve?

Respuesta

Una excepción puede ser la **causa** de otra cuando un error de bajo nivel provoca un error de más alto nivel. En Java, al crear una nueva excepción, se le puede pasar la excepción original como parámetro (el "cause"), creando así una cadena de causa-efecto. Esto permite conservar toda la información del error original mientras se presenta al llamador una excepción más significativa y adecuada a su nivel de abstracción.

A continuación se muestra un ejemplo donde una `IOException` (bajo nivel) se encapsula en una excepción personalizada de alto nivel:

```
// Excepción personalizada de alto nivel
public class ErrorCargaConfiguracion extends Exception {
    public ErrorCargaConfiguracion(String mensaje, Throwable causa) {
        super(mensaje, causa);
    }
}

// Clase que usa la excepción
public class Configuracion {

    public static String cargar(String ruta) throws
ErrorCargaConfiguracion {
        try {
            BufferedReader reader = new BufferedReader(new
FileReader(ruta));
            return reader.readLine();
        } catch (IOException e) {
            // Se encapsula la IOException como causa de la nueva
excepción
```

```
            throw new ErrorCargaConfiguracion("Error al cargar
configuración desde: " + ruta, e);
        }
    }

    public static void main(String[] args) {
        try {
            cargar("config.txt");
        } catch (ErrorCargaConfiguracion e) {
            e.printStackTrace();
        }
    }
}
```

Cuando esta excepción sale por pantalla mediante `printStackTrace()`, **sí se ve la causa**. La salida muestra la cadena completa de excepciones con la palabra clave `Caused by`:

```
ErrorCargaConfiguracion: Error al cargar configuración desde: config.txt
    at Configuracion.cargar(Configuracion.java:12)
    at Configuracion.main(Configuracion.java:18)
Caused by: java.io.FileNotFoundException: config.txt (No such file or
directory)
    at java.io.FileReader.<init>(FileReader.java:123)
    at Configuracion.cargar(Configuracion.java:9)
    ... 1 more
```

Este mecanismo es muy valioso porque permite a cada capa de la aplicación trabajar con excepciones de su propio nivel de abstracción, sin perder la información de diagnóstico del error original. El programador que recibe un `ErrorCargaConfiguracion` puede consultar `e.getCause()` para obtener la excepción original que provocó el problema.

Resumen de preguntas

Pregunta	Tema
1	Control de errores en C: valor especial vs. código de error con puntero
2	Definición de excepción y su objetivo
3	Ejemplo de `Calculadora` con `raiz` en Java usando `throw` y `try-catch`
4	Lanzar, capturar y propagar excepciones (con ejemplo de pila de llamadas)
5	Ventajas de la propagación automática frente a C
6	Excepciones como objetos + excepciones personalizadas
7	Información que lleva un objeto excepción: mensaje y stack trace
8	Múltiples bloques `catch` (solo se ejecuta uno)

Pregunta	Tema
9	Bloque `finally` con y sin `catch`
10	`finally` sin `catch`, con `return`, y cuándo se ejecuta
11	Excepciones controladas vs. no controladas + `RuntimeException`
12	`throws` como alternativa a capturar
13	Ejemplo con `throws FileNotFoundException` + `finally`
14	`throws` con excepciones no controladas
15	Cuándo usar cada tipo + comparativa entre lenguajes
16	Lanzar y relanzar excepciones dentro de `catch`
17	Causa (`Caused by`) y encapsulación de excepciones